Islamic University of Technology

Department of Electrical and Electronic Engineering

Project Submission

Monthly Dengue Case Prediction

Using Machine Learning and Deep Learning Models

Course Code: EEE 4710

Course Name: AI ML Lab

Submitted by

Name	Student ID
A.M. Tamim Haider	210021150
Tanvir Siddique	210021160
Marjukur Rajjak	210021321

April 19, 2026

Contents

1 Introduction **1**

1.1 Background of Dengue Disease 1

1.2 Importance of Dengue Case Forecasting 1

1.3 Dengue Situation in Brazilian States 1

1.4 Role of Machine Learning in Disease Prediction 2

1.5 Problem Statement 2

1.6 Project Objectives 3

2 Dataset Description **4**

2.1 Source of the Dataset 4

2.2 Description of the Dataset File 5

3 Data Preprocessing **7**

3.1 Importing Necessary Libraries 7

3.2 Loading the Dataset 7

3.3 Converting Date Column into Datetime Format 8

3.4 Checking Data Types 8

3.5 Checking Missing Values 9

3.6 Handling Missing Values Using Appropriate Imputation Strategy 9

3.7 Checking Duplicate Records 10

3.8 Basic Statistical Summary of the Dataset 10

4 Exploratory Data Analysis **11**

5 Feature Engineering **13**

5.1 Target Log Transformation 13

5.2 Dengue Lag Features 13

5.3 Rolling Window Features 13

5.4 Time and Seasonality Features 14

5.5 Climate Lag and Rolling Features 14

5.6 Population-Based Features 14

5.7 Weather Interaction Features 14

5.8 State/UF Encoding 14
5.9 Sequence Preparation for Deep Learning Models 15
5.10 Leakage Feature Removal 15

6 Train-Test Split 16

7 Machine Learning Model 18
7.1 Multilayer Perceptron (MLP) 18
7.1.1 Data Preprocessing and Preparation 18
7.1.2 How the MLP Processes the Data 19
7.1.3 Results and Performance 20
7.2 Graph Neural Network (GNN) and Hybrid Architecture 21
7.2.1 Data Preprocessing and Graph Construction 21
7.2.2 How the Network Processes the Data: Message Passing and Hybrid Fusion 22
7.2.3 Results and Performance 23
7.3 XGBoost Model 24
7.3.1 XGBoost Hyperparameters 24
7.3.2 Machine Learning Pipeline 25
7.3.3 Model Training Procedure 26
7.3.4 Model Testing and Results 26
7.3.5 Performance on Testing Data 27
7.3.6 Train-Test and Cross-Validation Performance Comparison 27
7.3.7 Final Model Performance Summary 27
7.4 Random Forest Model 28
7.4.1 Random Forest Hyperparameters 28
7.4.2 Machine Learning Pipeline 29
7.4.3 Model Training Procedure 30
7.4.4 Model Testing and Results 30
7.4.5 Performance on Testing Data 31
7.4.6 Train-Test and Cross-Validation Performance Comparison 31
7.4.7 Final Model Performance Summary 31
7.5 Bi-Directional LSTM 34
7.5.1 Bidirectional LSTM Hyperparameters 34
7.5.2 Machine Learning Pipeline 35
7.5.3 Model Training Procedure 35
7.5.4 Model Testing and Results 36
7.5.5 Performance on Testing Data 37
7.6 Train-Test Performance Comparison 37

7.7 Final Model Performance Summary 37
7.8 Temporal Fusion Transformer (TFT) 38
7.8.1 Temporal Fusion Transformer Hyperparameters 39
7.8.2 Model Training Procedure 39
7.8.3 Model Testing and Results 40
7.8.4 Performance on Testing Data 41
7.8.5 Train-Test Performance Comparison 41
7.8.6 Final Model Performance Summary 42

8 Comparison between Various Model Performance 43
8.1 Compare Performance of Various Models 43
8.2 Overall Model Performance Comparison 43
8.3 Comparative Analysis 44
8.4 Final Comparison Summary 44
8.5 Conclusion 45

1 Introduction

1.1 Background of Dengue Disease

Dengue is a mosquito-borne viral disease mainly spread by Aedes mosquitoes, especially Aedes aegypti. It is common in tropical and subtropical countries. Dengue can cause fever, headache, body pain, rash, and in severe cases, serious health complications.

The spread of dengue depends on weather and environmental conditions. Rainfall creates stagnant water where mosquitoes can breed. Temperature and humidity also affect mosquito growth and dengue transmission. For this reason, dengue cases often increase in certain seasons and in some regions more than others.

1.2 Importance of Dengue Case Forecasting

Dengue case forecasting means predicting the expected number of dengue cases in the future. This is important because early prediction can help health authorities take action before an outbreak becomes serious.

Accurate forecasting can help hospitals prepare medical resources, support mosquito control programs, and create awareness in high-risk areas. It can also help identify which states may face higher dengue cases in upcoming months.

1.3 Dengue Situation in Brazilian States

Each country has different climate, population, and geographic conditions across its states. Because of these differences, dengue cases are not the same in all regions. Some states may have higher cases due to weather, urbanization, population density, and mosquito breeding conditions.

The dataset used in this project contains monthly dengue case information for Brazilian states. It includes dengue case counts, weather-related variables, population data, geographic information, and socioeconomic features. This makes the dataset useful for studying dengue patterns over time and across different states.

Since dengue cases vary by month and location, it is important to consider both time-based features and state-based features. In this project, the Brazilian state code and

monthly time information are used along with climate and population features to predict future dengue case counts.

1.4 Role of Machine Learning in Disease Prediction

Machine Learning can learn patterns from past data and use those patterns to make future predictions. In disease prediction, Machine Learning helps find the relationship between disease cases and important factors such as weather, seasonality, population, and previous disease trends.

For dengue prediction, this relationship can be complex. Dengue cases may increase or decrease depending on rainfall, temperature, humidity, population size, month of the year, and previous dengue case numbers. Simple methods may not always capture these complex patterns properly. For this reason, different Machine Learning and Deep Learning models are used in this project.

In this project, six different models are used to predict monthly dengue cases in Brazilian states. These models are XGBoost, Random Forest, MLP, GNN, Bi-LSTM, and TFT. XGBoost and Random Forest are tree-based Machine Learning models that work well with tabular data. MLP is a neural network model that can learn nonlinear relationships from input features. GNN can learn relationships between different Brazilian states if the data is represented as a graph. Bi-LSTM is useful for learning time-dependent patterns from past monthly data. TFT is an advanced time-series forecasting model that can handle temporal features and important input variables.

By using these models, the project can compare different approaches and identify which model performs better for dengue case forecasting. This helps to understand whether traditional Machine Learning models or advanced Deep Learning models are more suitable for predicting monthly dengue cases.

.

1.5 Problem Statement

Dengue is a serious public health problem in many regions, including Brazil. The number of dengue cases changes over time and varies across states. These changes are influenced by past dengue trends, weather conditions, seasonal effects, population size, and other regional characteristics.

The main problem is that dengue outbreaks can occur suddenly, and without proper prediction, health authorities may not have enough time to prepare. Therefore, there is a

need for a data-driven forecasting approach that can estimate future dengue case counts using historical and environmental information.

This project focuses on developing a Machine Learning model that can predict the expected monthly dengue case count for each Brazilian state. The model uses previous dengue cases, climate variables, seasonality, and population-related features to make predictions.

1.6 Project Objectives

The main objective of this project is to build an XGBoost Machine Learning model to predict monthly dengue cases in Brazilian states.

The specific objectives are:

1. To load and analyze the dengue dataset.
2. To understand the number of samples, features, and data types.
3. To preprocess the data and handle missing values.
4. To create useful features such as lag values, rolling averages, seasonal encoding, and population-related features.
5. To remove leakage features from the dataset.
6. To split the data into training and testing sets.
7. To train an XGBoost regression model.
8. To evaluate the model using MAE, RMSE, and R^2 score.
9. To visualize actual and predicted dengue cases.
10. To compare train-test results with cross-validation results.

2 Dataset Description

The dataset used in this project is a monthly dengue dataset of Brazilian states. It contains dengue case records along with weather, population, geographic, land-cover, and socioeconomic information. The main goal of using this dataset is to predict the expected monthly dengue case count for each Brazilian state.

The dataset is suitable for this project because dengue cases are affected by many factors such as rainfall, temperature, humidity, seasonality, population size, and previous dengue trends.

2.1 Source of the Dataset

The dataset file used in this project is named:

```
Brazil_UF_dengue_monthly.csv
```

The main source of this dataset-

Primary Journal Publication (2024)

- Title: A reproducible ensemble machine learning approach to forecast dengue outbreaks
- Authors: Alessandro Sebastianelli, Dario Spiller, Raquel Carmo, James Wheeler, Artur Nowakowski, Ludmilla Viana Jacobson, Dohyung Kim, Hanoch Barlevi, Zoraya El Raiss Cordero, Felipe J. Colón-González, Rochelle Schneider.
- Journal: Nature Scientific Reports (2024)
- Summary: This paper details the full methodology of combining meteorological data (like NDVI, temperature, dewpoint) with socio-economic vulnerability indices (like the IVS and IDHM found in your dataset) to predict dengue incidence rates at the state level (UF) in Brazil one month in advance.
- Link: Read on PubMed Central (PMC10869339)

2.2 Description of the Dataset File

The dataset is stored in CSV format and represents a monthly panel time-series dataset. Each row contains dengue-related information for a specific Brazilian state in a specific month.

The dataset covers January 2001 to December 2019, giving 19 years of continuous monthly records across all 27 Brazilian states. Each state has 228 monthly records, so the total number of samples is 6,156. The dataset contains 62 columns, including dengue case counts, climate variables, population information, environmental features, and state-level characteristics.

Table 2.1: *Dataset File Description*

Item	Value
Number of samples/rows	6,156
Number of columns	62
Number of Brazilian states	27
Time period	2001–2019
Records per state	228 months
Missing values	0
Duplicate rows	0

The dataset is clean and well-structured because it has no missing values and no duplicate records. This is important because the model performance will depend mainly on the actual data pattern rather than missing value handling or imputation.

Some important columns in the dataset are Date, Year, Month, CD_UF, cases_total, temperature, humidity, precipitation, population, area, forest cover, and urban cover. These features are useful because dengue transmission is affected by weather conditions, population density, environmental factors, urban infrastructure, and seasonal variation.

The target variable of this project is cases_total, which represents the total number of dengue cases in a specific Brazilian state for a specific month.

Table 2.2: *Target Variable Statistical Summary*

Statistic	Value
Minimum cases	0
Median cases	360.5
Mean cases	2094.24
Maximum cases	251,725
Standard deviation	Approximately 8,800

The target variable shows large variation. Some months have very few dengue cases, while some outbreak months have extremely high case counts. This makes the prediction task a challenging regression problem. To reduce the effect of very large outbreak values, log transformation is applied using:

$$\log(1 + cases_total)$$

This transformation makes the target distribution more stable and helps improve model training.

The dataset also includes age-based dengue case columns such as cases0_19 and cases20_99, which represent dengue cases for age groups 0–19 and 20–99. These columns provide useful demographic information, but they should not be used as input features for prediction because they are directly related to cases_total. Using them may cause data leakage and produce unrealistically high model performance.

The feature space mainly covers three important dengue-related domains:

- Vector biology: Temperature, precipitation, humidity, dewpoint, and NDVI help describe mosquito breeding conditions. Rainfall creates stagnant water, humidity supports larval survival, and vegetation provides suitable mosquito habitats.
- Urban infrastructure: Features such as sanitation access, waste disposal, urban cover percentage, and infrastructure vulnerability explain how human-made environments can increase mosquito breeding sites.
- Host availability: Population density, urban-rural distribution, and HDI-related features explain human exposure and transmission risk.

Overall, this dataset is suitable for monthly dengue case prediction because it combines long-term dengue history, climate conditions, population data, environmental variables, seasonality, and state-level characteristics across all Brazilian states.

3 Data Preprocessing

3.1 Importing Necessary Libraries

At the beginning of the project, the required Python libraries were imported. The main libraries used were Pandas and NumPy for data handling and numerical operations. Matplotlib was used for visualization. For machine learning, libraries from Scikit-learn were used for preprocessing, train-test splitting, pipeline creation, imputation, scaling, and model evaluation. The XGBoost library was also imported to train the XGBoost regression model.

These libraries helped to load the dataset, clean the data, prepare features, train the model, and evaluate its performance.

3.2 Loading the Dataset

The dataset was loaded from the CSV file named:

```
Brazil_UF_dengue_monthly.csv
```

The dataset was imported using Pandas. After loading the file, the first few rows were displayed to understand the structure of the data. The dataset contains monthly dengue-related information for Brazilian states.

The initial shape of the dataset was:

Table 3.1: *Initial Dataset Shape*

Item	Value
Number of rows	6156
Number of columns	62

This means the dataset contains 6156 monthly records and 62 variables.

3.3 Converting Date Column into Datetime Format

The Date column was converted into datetime format. This step is important because the project deals with monthly dengue forecasting, so time information must be handled properly.

After converting the date column, year, month, and quarter information were extracted from it. These time-based features are useful because dengue cases often show seasonal patterns.

For example, the following features were created:

- Year
- Month
- Quarter

These features help the model understand how dengue cases change over time

3.4 Checking Data Types

The data types of all columns were checked. This step is necessary because machine learning models require properly formatted input data.

The dataset mainly contains numerical columns. The Date column was originally stored as an object type and later converted into datetime format.

The dataset contains:

Table 3.2: *Dataset Data Types*

Data Type	Description
Numerical features	Dengue cases, climate data, population data, geographic values, socioeconomic indicators
Categorical feature	Brazilian state code CD_UF
Date feature	Monthly date information

Although CD_UF is stored as a number, it represents a state category. Therefore, it was treated as a categorical feature during model preparation.

3.5 Checking Missing Values

Missing values were checked before model training. The original dataset did not contain major missing value problems. However, after feature engineering, some missing values were created naturally.

This happened because lag and rolling features use previous months' data. For example, the first month of each state cannot have a previous month value, so casesLag1 becomes missing for that row.

Examples of features that may create missing values are:

- casesLag1
- casesLag2
- casesLag3
- casesRollingMean3
- casesRollingStd3
- temperatureLag1
- precipitationLag1

These missing values are normal in time-series feature engineering.

3.6 Handling Missing Values Using Appropriate Imputation Strategy

Missing values were handled using an imputation strategy inside the machine learning pipeline.

For numerical features, missing values were replaced using the median value. Median imputation was selected because it is less affected by extreme values. This is useful for dengue data because dengue cases can have sudden outbreak peaks.

For categorical features, missing values were replaced using the most frequent value.

The preprocessing pipeline used:

Table 3.3: *Missing Value Imputation Strategy*

Feature Type	Imputation Strategy
Numerical features	Median imputation
Categorical features	Most frequent imputation

This approach is better than manually filling missing values before splitting because the imputer is fitted only on the training data inside the pipeline. This helps prevent data leakage.

3.7 Checking Duplicate Records

The dataset was also checked for duplicate records. Duplicate records can affect model training because they may give extra importance to repeated observations.

After checking the dataset, no duplicate records were found. Therefore, no duplicate rows needed to be removed.

3.8 Basic Statistical Summary of the Dataset

A basic statistical summary was generated to understand the distribution of the numerical columns. This included values such as mean, standard deviation, minimum, maximum, and quartiles.

The target variable of this project is:

```
cases_total
```

This column represents the monthly dengue case count. The target variable showed a large variation, where some months had very low dengue cases and some months had very high outbreak values.

Because of this large variation, the target variable was transformed using:

$$\log 1p(cases_total)$$

This transformation reduces the effect of very large dengue outbreak values and makes the model training more stable.

4 Exploratory Data Analysis

Exploratory Data Analysis, or EDA, was performed to understand the main patterns of dengue cases before model training. It helped identify how dengue cases change over time, how they differ across Brazilian states, and how climate, population, and seasonal factors are related to dengue outbreaks.

Key findings from the EDA are following-

- Monthly trend: Dengue cases do not remain constant over time. Some months and years show sudden increases, which indicates outbreak-type behavior.
- State-wise variation: Dengue cases differ across Brazilian states. More populated and urbanized states generally show higher dengue case counts.
- Seasonal pattern: Dengue cases are higher in some months, especially around February, March, April, and May. Lower cases are mostly observed around August to November. This shows that dengue has a strong seasonal pattern.
- Rainfall effect: Same-month rainfall has weak direct correlation with dengue cases. However, rainfall can affect dengue after a delay because mosquito breeding takes time. Therefore, lag and rolling rainfall features were created.
- Temperature effect: Temperature alone does not strongly explain dengue cases. Dengue transmission depends on a combination of temperature, rainfall, humidity, population, and previous dengue trends.
- Population effect: Population-related features show a positive relationship with dengue cases. States with larger populations may report more dengue cases because more people are exposed.
- Correlation analysis: Some features have stronger correlation with the target variable cases_total, especially population-related features. However, cases0_19 and cases20_99 were excluded because they are directly related to the target and may cause data leakage.
- Weak correlation features: Some variables, such as precipitation, wind, dewpoint, area, and unemployment-related features, show weak direct correlation with dengue cases. They were not removed immediately because Random Forest and XGBoost can capture nonlinear relationships and feature interactions.

Overall, EDA showed that dengue case prediction depends on time trend, seasonality, state-level differences, population, climate conditions, and previous dengue patterns. This helped guide the feature engineering and model training process.

5 Feature Engineering

Feature engineering was performed to prepare the dengue dataset for six different machine learning and deep learning models: XGBoost, Random Forest, MLP, GNN, Bi-LSTM, and TFT. Since these models use data in different ways, the raw dataset was transformed into meaningful features that can represent dengue trends, climate effects, seasonality, population influence, and state-level differences.

5.1 Target Log Transformation

The target variable of this project is cases_total, which represents the monthly dengue case count. Since dengue cases can sometimes become very large during outbreaks, the target variable was transformed using log1p(cases_total). This helps reduce the effect of extreme values and makes model training more stable. After prediction, expm1() can be used to convert the result back to real dengue case numbers.

5.2 Dengue Lag Features

Lag features were created from previous dengue case values. These features help the model learn how past dengue cases affect future dengue cases. For example, previous month, previous two-month, previous three-month, six-month, and twelve-month dengue cases can be used as input features. These are very important because dengue outbreaks usually continue over time instead of appearing randomly.

5.3 Rolling Window Features

Rolling window features were created to capture recent dengue trends. Rolling mean and rolling standard deviation over 3-month, 6-month, or 12-month windows help show whether dengue cases are increasing, decreasing, or fluctuating. These features are useful for detecting outbreak patterns.

5.4 Time and Seasonality Features

Dengue cases are strongly seasonal. Therefore, time-based features such as year, month, and quarter were extracted from the date column. Month and quarter were also converted into cyclical features using sine and cosine encoding. This helps the model understand the repeating seasonal pattern of dengue.

5.5 Climate Lag and Rolling Features

Weather conditions can affect dengue transmission, but the effect may appear after some delay. For this reason, lag and rolling features were created from climate variables such as rainfall, temperature, humidity, and dewpoint temperature. These features help the model capture delayed climate effects on mosquito breeding and dengue spread.

5.6 Population-Based Features

Population-related features were included because states with larger populations may report more dengue cases. Features such as total population, population density, urbanization ratio, and incidence rate per 100,000 people help the model understand dengue risk more fairly across different states.

5.7 Weather Interaction Features

Dengue transmission may depend on the combined effect of different weather factors. Therefore, interaction features such as rainfall-temperature interaction and humidity-temperature interaction were created. These features help the model learn how multiple climate variables work together.

5.8 State/UF Encoding

The Brazilian state code CD_UF was used as a state-level feature. Dengue patterns can vary from one state to another because of climate, population, geography, and urbanization. For XGBoost, Random Forest, and MLP, this feature can be encoded as a categorical variable. For GNN, each state can be treated as a node in a graph.

5.9 Sequence Preparation for Deep Learning Models

For Bi-LSTM and TFT models, the dataset was arranged in time order for each state. A fixed number of previous months can be used as input to predict the next month's dengue cases. This allows the models to learn time-dependent patterns from dengue history, weather, and seasonal features.

5.10 Leakage Feature Removal

Data leakage was avoided by removing features that directly reveal the target value. The current month cases_total, transformed target column, and direct current-month dengue case breakdowns should not be used as input. Only past dengue information, climate features, time features, population features, and state-level features were used for model training.

6 Train-Test Split

Train-test splitting was performed to separate the dataset into two parts: one part for training the models and another part for testing their performance.

The training data is used to teach the models the relationship between dengue cases and different features such as previous dengue trends, rainfall, temperature, population, and seasonality. The testing data is used to evaluate how well the trained models can predict dengue cases on unseen future data.

Since this project is based on monthly dengue forecasting, the dataset has a time-series nature. In time-series problems, data order is important because past records are used to predict future values. Therefore, a random train-test split was not used.

Instead, a time-based train-test split was applied. The earlier records were used for training, and the later records were used for testing. This is more realistic because, in real forecasting, future dengue cases cannot be known during model training.

A 75% training and 25% testing split was used.

Table 6.1: *Training and Testing Dataset Split*

Dataset Part	Percentage	Records per State	Approximate Time Period
Training set	75%	171 months	January 2001 – March 2015
Testing set	25%	57 months	April 2015 – December 2019

The training dataset contains the earlier dengue records from all 27 Brazilian states. This part helps the models learn long-term patterns, seasonal behavior, climate effects, and previous dengue trends.

The testing dataset contains the later records from all 27 states. This part is used to check whether the models can predict dengue cases for future months.

Random splitting was avoided because it could mix future records with past records. This would create future data leakage, where the model indirectly learns information from future months during training. As a result, the model performance would look better than it actually is.

Overall, the time-based train-test split makes the evaluation more realistic and suitable

for monthly dengue case forecasting.

7 Machine Learning Model

In this project, six different machine learning models will be used to predict monthly dengue cases from the dataset. The main objective is to compare their performance and identify which model can forecast dengue cases more accurately.

The models will learn patterns from historical dengue records, climate variables, population information, seasonal features, and state-level characteristics. Since dengue cases depend on many factors such as previous case trends, rainfall, temperature, humidity, and population density, using multiple models helps evaluate the problem from different learning approaches.

7.1 Multilayer Perceptron (MLP)

A Multilayer Perceptron (MLP) is used as one of the main deep learning models for monthly dengue case prediction. It is a feed-forward neural network that learns the relationship between input features and the target variable.

In this project, each row of the dataset represents one Brazilian state in one specific month. The MLP takes the processed feature values, such as climate, population, vegetation, socioeconomic conditions, seasonality, and previous dengue trends, and predicts the monthly dengue case count.

7.1.1 Data Preprocessing and Preparation

Before the data could be ingested by the Multilayer Perceptron (MLP), it required rigorous preprocessing to ensure the model could effectively extract meaningful patterns. The dataset consists of a continuous 19-year monthly record (2001–2019) across 27 Brazilian states, yielding a rich, zero-sparsity matrix of 62 distinct features per time step.

The data processing pipeline for the MLP involved the following key steps:

- Feature Aggregation and Vectorization: The 62 features—spanning epidemiological (age cohorts), climatic, vegetative, and socioeconomic domains—were consolidated into independent feature vectors. For the standalone MLP, each row (representing a specific state at a specific month) was treated as an independent observation.
- Standardization of High-Variance Features: To prevent gradient explosion and ensure

stable convergence during training, the input features were standardized (using Z-score normalization / StandardScaler). Rather than using multiple or mixed scaling methods, this single standardization approach centered the diverse data signals—like temperature readings and population figures—around a mean of zero with a standard deviation of one, placing them onto a mathematically comparable scale.

- Tensor Conversion and Batching: Finally, the structured, standardized tabular data was converted into PyTorch tensors. The dataset was split into training, validation, and testing sets to evaluate the model's ability to generalize to unseen temporal data. These tensors were then wrapped into PyTorch DataLoader objects to facilitate efficient, batched feeding into the neural network.

7.1.2 How the MLP Processes the Data

The standalone MLP is designed to capture state-level temporal dynamics and feature interactions, evaluating the outbreak risk of a state based purely on its internal, localized data at a given time.

Once the processed data is fed into the network, the PyTorch-based MLP processes it through the following flow:

- Input Layer Configuration: The network takes an input vector of size 62 (representing all features for a single state-month instance). Unlike a Graph Neural Network (GNN), the MLP is structurally "blind" to spatial relationships; it does not know what is happening in neighboring states. It relies entirely on the richness of the localized data.
- Hidden Layers and Non-Linear Transformations (PyTorch implementation): The data passes through a neural network class built using torch.nn.Module. This consists of a series of fully connected layers defined by nn.Linear. During the forward pass, the network applies non-linear activation functions, specifically nn.ReLU(), after each linear transformation. This is where the core learning happens: the model learns complex, non-linear correlations between the independent variables, figuring out how specific combinations of high temperatures, high vegetation indices, and demographic vulnerabilities converge to trigger an outbreak.
- Weight Optimization (State-Level Dynamics): As the data flows through the network, the weights are multiplied and biases are added. During training, the model calculates the error (the difference between its prediction and the actual case count) using a loss function, specifically Mean Squared Error (nn.MSELoss). It then uses backpropagation via the Adam optimizer (torch.optim.Adam) to iteratively adjust these weights and minimize the error.

- Regression Output: The final nn.Linear layer of the network condenses the learned abstractions into a single continuous output node. This node applies a final linear transformation to output the predicted raw dengue case count for that specific state and month.

7.1.3 Results and Performance

The standalone MLP demonstrated exceptionally strong predictive capabilities. When evaluated on the testing data, the model achieved an impressive R^2 score of 0.9194.

This high level of performance leads to several important conclusions:

- Strong Localized Signals: The fact that the MLP captured nearly 92% of the variance in dengue outbreaks proves that the heavily engineered, 62-feature inputs (climate, vegetation, demographics) contain a highly predictive, self-contained signal. The model successfully mapped the internal state conditions to outbreak severity.
- Baseline for Advanced Architectures: Despite its high accuracy, the standalone MLP is theoretically bounded by its spatial blindness. It treats a state completely isolated from its geographic neighbors.
- The Need for Spatial Reasoning: Dengue is an infectious disease driven by vectors (mosquitoes) and human movement, meaning it naturally crosses borders. The remaining unexplained variance in the MLP points to this missing spatial dynamic. While the MLP excels at understanding when a state's internal conditions are right for an outbreak, it cannot predict an outbreak spilling over from a neighboring state—setting the perfect stage for the introduction of Graph Neural Networks (GNNs).

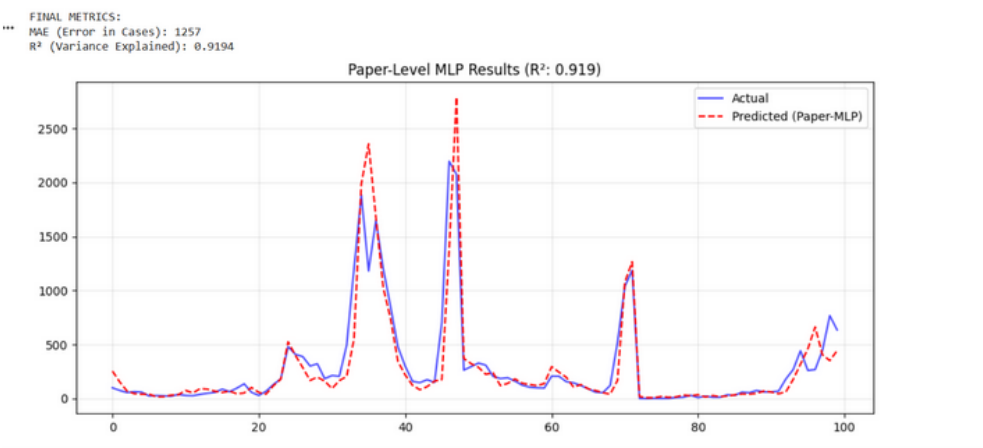


Figure 7.1

7.2 Graph Neural Network (GNN) and Hybrid Architecture

In this project, a Graph Neural Network (GNN) is used to capture the spatial relationship between Brazilian states. Dengue does not only depend on the internal condition of one state; outbreaks can also be influenced by neighboring states due to human movement, geographic proximity, and mosquito-related transmission patterns.

For the GNN, each of the 27 Brazilian states is treated as a node in the graph. The features of each state, such as climate, population, vegetation, and socioeconomic variables, are used as node features. The connections between neighboring states are represented as edges, allowing the model to learn spatial dengue spread.

A Hybrid GNN + MLP architecture is used to combine two types of information:

- GNN part: Learns spatial relationships between neighboring states.
- MLP part: Learns the local dengue pattern of each state from its own features.
- Fusion layer: Combines both outputs to make the final dengue case prediction.

This hybrid structure is useful because dengue forecasting depends on both local factors and regional spread. The MLP captures state-specific trends, while the GNN captures the effect of nearby states. Together, the hybrid model provides a stronger and more realistic approach for monthly dengue case prediction.

7.2.1 Data Preprocessing and Graph Construction

While the MLP required tabular, isolated feature vectors, the Graph Neural Network (GNN) requires the data to be formatted relationally to understand spatial context. Dengue spreads through biological vectors (mosquitoes) and human mobility, meaning an outbreak in one state heavily influences its geographic neighbors.

To process the data for the GNN, the standardized tabular data was transformed into graph structures (e.g., using PyTorch Geometric Data objects):

- Defining Nodes (Vertices): The 27 Brazilian states were initialized as individual nodes in the graph. The rich, standardized 62-feature vectors (containing climate, vegetation, demographic, and socioeconomic data) were assigned as the feature matrix (x) for these nodes at each time step.

- Defining Edges (Connectivity): To capture spatial disease transmission, edges (edge_index) were defined between the nodes. These edges represent the spatial borders and geographic proximity between the Brazilian states, effectively mapping the physical avenues through which the outbreak could spread.
- Feature Richness as a Prerequisite: A critical part of the GNN data preparation was ensuring high feature quality. As noted in our project analysis, a GNN running on weak features merely propagates noise across the graph. The zero-sparsity, 62-feature inputs gave each node meaningful, high-density information to communicate.

7.2.2 How the Network Processes the Data: Message Passing and Hybrid Fusion

Dengue operates through two independent mechanisms: spatial spread across borders and state-level temporal dynamics. No single architecture captures both perfectly. A standalone MLP is structurally blind to neighbor outbreaks, while a standalone GNN risks over-smoothing, diluting an individual state's unique seasonality into a generic neighborhood average during data aggregation.

To solve this, the data is processed through a Hybrid GNN+MLP Architecture:

- The GNN Path (Message Passing): The graph data passes through GNN layers where "message passing" occurs. In this process, each state (node) looks at its connected neighbors, aggregates their 62-feature risk profiles, and updates its own state representation. If a neighboring state has optimal climate conditions for mosquitoes and a rising case count, the GNN mathematically passes that "spatial threat" across the border.
- The MLP Path (Temporal Dynamics): In parallel, the original state data is fed through a standard Multilayer Perceptron (as described in the MLP documentation). This path explicitly ignores the neighbors and focuses 100% on the localized, internal conditions of the state to preserve its unique temporal seasonality.
- The Fusion Layer: The outputs of both the GNN (spatial context) and the MLP (local context) are concatenated and passed into a final neural network fusion layer. During training, backpropagation teaches this fusion layer how to dynamically balance the two signals. For any given sample, the network learns whether to weigh the internal state dynamics more heavily or to sound the alarm based on spatial spread from a neighbor.

7.2.3 Results and Performance

The Hybrid GNN+MLP model successfully united localized reasoning with spatial awareness, achieving an outstanding R^2 score of 0.9464.

This performance (an improvement over the standalone MLP's 0.9194) confirms several structural hypotheses:

- Complementary Signals: The jump in accuracy empirically validates that graph reasoning added a genuine, complementary signal. The MLP handled the when (local seasonality), while the GNN handled the where (spatial spillover).
- Prevention of Signal Dilution: By running the architectures in parallel rather than relying purely on a deep GNN, the model successfully avoided washing out state-level nuances. It proved that a dynamic fusion layer is highly effective at balancing macro-regional spread with micro-regional climate conditions.
- The Value of Rich Data: The strong performance of the hybrid model was largely made possible by the dataset's depth. Because every node was armed with robust climate, vegetation, and socioeconomic signals, the GNN's message passing circulated genuine epidemiological risk factors rather than arbitrary noise, making the graph logic highly effective.

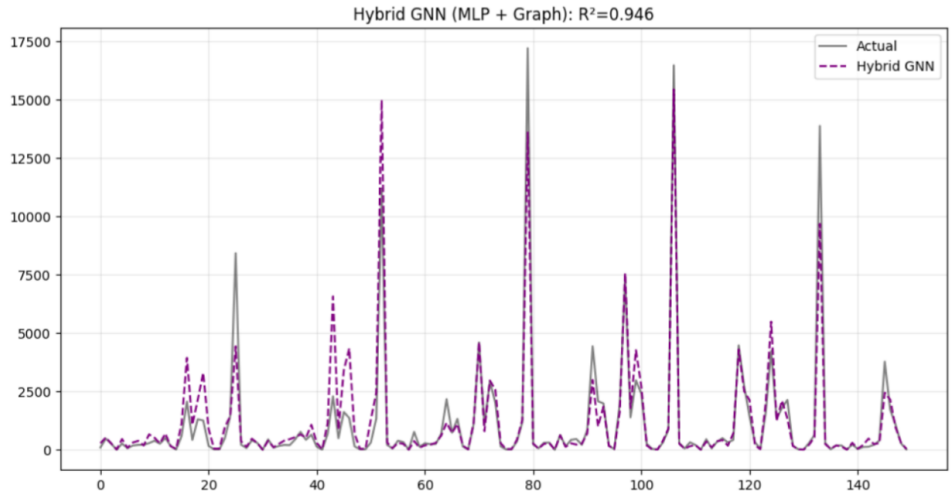


Figure 7.2

7.3 XGBoost Model

XGBoost Regressor was used for monthly dengue case forecasting because the dataset is mainly structured tabular data with dengue history, climate variables, seasonality, population information, and state-level features. XGBoost is suitable for this type of problem because it can learn nonlinear relationships and complex feature interactions without needing a deep sequential architecture.

The target variable was cases_total, but the model was trained using log1p(cases_total). This log transformation reduced the effect of very large dengue outbreak values and made the regression problem more stable. After prediction, the output was converted back to real dengue case counts using inverse exponential transformation.

The model used engineered lag features such as previous 1, 2, 3, 6, and 12-month dengue cases. Rolling mean and rolling standard deviation features were also created to capture short-term and long-term dengue trends. Seasonal features such as month, quarter, sine, and cosine encodings helped the model understand yearly dengue cycles. Climate features, climate lag values, rolling climate statistics, population-based features, and weather interaction features were also included.

Overall, XGBoost was suitable for this project because dengue cases depend on previous outbreak patterns, seasonal variation, weather conditions, and state-level differences. The model can combine these factors effectively and produce a single continuous prediction for the expected monthly dengue case count.

7.3.1 XGBoost Hyperparameters

The XGBoost model was trained using the XGBRegressor with the objective function set to reg:squarederror, which is appropriate for regression problems. The random seed was fixed at 42 to make the result reproducible.

Hyperparameter tuning was performed using RandomizedSearchCV with time-series cross-validation. The model was tested using different values of tree number, tree depth, learning rate, subsampling ratio, column sampling ratio, child weight, and regularization parameters.

The best XGBoost hyperparameters were:

Table 7.1: *Best XGBoost Hyperparameters*

Hyperparameter	Selected Value
Number of estimators	900
Maximum depth	5
Learning rate	0.05
Subsample	0.9
Column sample by tree	1.0
Minimum child weight	3
Alpha regularization	0.01
Lambda regularization	2
Objective	reg:squarederror
Tree method	hist

The best cross-validation score during tuning was -0.3945 using negative mean absolute error as the scoring function. This indicates that the selected parameter combination gave the lowest average validation error among the tested settings.

7.3.2 Machine Learning Pipeline

The machine learning pipeline started by loading the monthly dengue dataset for 27 Brazilian states from 2001 to 2019. The Date column was converted into datetime format, and the data was sorted by state code and date to preserve the time-series order.

Feature engineering was then applied to create dengue lag features, rolling statistics, climate lag features, climate rolling statistics, seasonal encodings, population-based features, and weather interaction features. Current-month dengue case values were removed from the input features to avoid data leakage.

The final feature set contained 70 selected features. Among them, 69 were numerical features and CD_UF was treated as a categorical state feature. Numerical features were processed using median imputation and standard scaling. The categorical state code was processed using most-frequent imputation and one-hot encoding.

A chronological 75% training and 25% testing split was used instead of a random split. This is important because dengue forecasting is time-dependent. The split date was April 2015. The training set contained 4,617 samples, while the testing set contained 1,539 samples.

7.3.3 Model Training Procedure

The XGBoost model was trained on the log-transformed dengue case target. Time-series cross-validation with 4 splits was used during hyperparameter tuning so that each validation fold occurred after its corresponding training fold. This preserved the natural time order of the forecasting problem.

The model learned from previous dengue case trends, rolling outbreak behavior, climate conditions, seasonality, and state-level population features. Since XGBoost builds boosted decision trees sequentially, each new tree tries to correct the errors made by previous trees. This allows the model to improve prediction accuracy step by step.

The model used 900 boosted trees with a maximum depth of 5. This gave the model enough flexibility to learn nonlinear dengue patterns while controlling overfitting through regularization, subsampling, and limited tree depth.

7.3.4 Model Testing and Results

After training, the best XGBoost model was selected based on the lowest validation error from hyperparameter tuning. The trained model was then applied to the unseen chronological test set.

The model predicted dengue cases in log-transformed scale first. Then the predicted values were converted back into actual case counts using inverse transformation. Any negative predicted value after inverse transformation was clipped to zero because dengue case counts cannot be negative.

On the test set, XGBoost achieved a log-scale R^2 score of 0.9317. This means the model explained more than 93% of the variation in dengue cases in log-transformed space. In real case-count scale, the model achieved an MAE of about 1,110 cases and an RMSE of about 6,038 cases.

The model also performed well in outbreak detection. Using an outbreak threshold of 1,106 cases, XGBoost achieved an MCC score of 0.8904, precision of 0.8981, recall of 0.9550, and F1-score of 0.9257. The high recall shows that the model was strong at detecting outbreak-level months.

7.3.5 Performance on Testing Data

Table 7.2: *XGBoost Performance on Testing Data*

Metric	XGBoost Result
R^2 Score in Log Scale	0.9317
Mean Absolute Error	1,110 cases
Root Mean Squared Error	6,038 cases
MAPE	52.79%
WAPE	37.10%
Accuracy-like Percentage	62.90%
MCC for Outbreak Detection	0.8904
Precision	0.8981
Recall	0.9550
F1-score	0.9257

The log-scale R^2 value shows very strong predictive performance. However, the real-scale error values are larger because dengue outbreaks can contain very high case counts, and inverse transformation magnifies prediction errors for large outbreak months.

7.3.6 Train-Test and Cross-Validation Performance Comparison

The XGBoost model showed stable performance in both train-test evaluation and time-series cross-validation. In the final train-test split, the model achieved a log-scale R^2 of 0.9317, MAE of 1,110 cases, and RMSE of 6,038 cases.

In 4-fold time-series cross-validation, the average XGBoost performance was also strong, with a mean log-scale R^2 of 0.9292, mean MAE of 939 cases, and mean RMSE of 5,136 cases. The closeness between the cross-validation result and the final test result indicates that the model generalized well and did not depend only on one specific split.

7.3.7 Final Model Performance Summary

The XGBoost Regressor achieved strong performance for monthly dengue case forecasting in Brazilian states. It obtained a test-set log-scale R^2 of 0.9317, MAE of 1,110 cases, RMSE of 6,038 cases, and WAPE of 37.10%. For outbreak detection, it achieved an MCC of 0.8904 and an F1-score of 0.9257.

Feature importance analysis showed that casesLag1 was the most important predictor,

meaning the previous month's dengue case count had the strongest influence on the next month's prediction. Seasonal features, state-level encoding, climate interaction, and population-related features also contributed to the model.

Overall, XGBoost performed very well because it effectively used lagged dengue history, seasonal patterns, climate variables, and state-level differences. It provides a strong machine learning baseline for dengue forecasting and can be compared with Random Forest and deep learning models such as Bi-LSTM and Temporal Fusion Transformer.

7.4 Random Forest Model

Random Forest Regressor was used for monthly dengue case forecasting because it is a strong machine learning model for structured tabular data. The dataset contains dengue history, climate variables, seasonal information, population features, and state-level information. Random Forest can handle these mixed relationships effectively by combining many decision trees.

The target variable was cases_total, but the model was trained using log1p(cases_total). This transformation reduced the effect of very large dengue outbreak values and made the model training more stable. After prediction, the output was converted back into real dengue case counts using inverse exponential transformation.

Random Forest works by training many decision trees on different random samples of the data. Each tree gives a prediction, and the final output is calculated by averaging the predictions of all trees. This helps reduce overfitting and makes the model more stable than a single decision tree.

The model used dengue lag features, rolling case statistics, seasonal encodings, climate variables, climate lag features, rolling climate statistics, population features, and weather interaction features. These features allowed the model to learn previous outbreak behavior, seasonal dengue patterns, and the effect of climate conditions.

Overall, Random Forest was suitable for this dengue forecasting task because it can capture nonlinear relationships, handle many input features, and provide stable predictions for monthly dengue case counts.

7.4.1 Random Forest Hyperparameters

The Random Forest model was trained using RandomForestRegressor. The random seed was fixed at 42 to make the result reproducible. Hyperparameter tuning was performed using RandomizedSearchCV with time-series cross-validation.

The model was tested using different values of the number of trees, maximum tree depth, minimum samples required for splitting, minimum samples at leaf nodes, and maximum number of features considered at each split.

The best Random Forest hyperparameters were:

Table 7.3: *Best Random Forest Hyperparameters*

Hyperparameter	Selected Value
Number of estimators	700
Maximum depth	12
Minimum samples split	5
Minimum samples leaf	2
Maximum features	0.8
Random state	42

The best cross-validation score during tuning was -0.3907 using negative mean absolute error as the scoring function. This means the selected parameter combination gave the lowest average validation error among the tested Random Forest settings.

7.4.2 Machine Learning Pipeline

The machine learning pipeline started by loading the monthly dengue dataset for 27 Brazilian states from 2001 to 2019. The Date column was converted into datetime format, and the data was sorted by Brazilian state code and date to preserve chronological order.

Feature engineering was applied to create dengue lag variables, rolling statistics, climate lag variables, climate rolling statistics, seasonal features, population-based features, and weather interaction features. The current-month dengue case value was removed from the input features to prevent data leakage.

The final feature set contained 70 selected features. Numerical features were processed using median imputation and standard scaling. The categorical state code CD_UF was processed using most-frequent imputation and one-hot encoding.

A chronological 75% training and 25% testing split was used instead of a random split. This was important because dengue forecasting is a time-dependent problem. The training set contained 4,617 samples, while the testing set contained 1,539 samples. The test period began from April 2015 onward.

7.4.3 Model Training Procedure

The Random Forest model was trained on the log-transformed dengue case target. Time-series cross-validation with 4 splits was used during hyperparameter tuning so that future data was not used to predict past data.

During training, the model created 700 decision trees. Each tree learned slightly different patterns from the training data because Random Forest uses randomness in data sampling and feature selection. The final prediction was obtained by averaging the outputs of all trees.

The maximum depth was limited to 12 to control overfitting. The minimum samples split was set to 5, and the minimum samples leaf was set to 2, which helped prevent the trees from becoming too specific to the training data. The maximum feature ratio was set to 0.8, meaning each tree split considered only part of the available features.

This training strategy allowed the model to learn stable relationships between previous dengue cases, seasonal effects, climate variables, population features, and future dengue case counts.

7.4.4 Model Testing and Results

After training, the best Random Forest model was selected based on the lowest validation error from hyperparameter tuning. The trained model was then applied to the unseen chronological test set.

The model first predicted dengue cases in log-transformed scale. Then the predictions were converted back into actual dengue case counts using inverse exponential transformation. Negative predicted values were clipped to zero because dengue case counts cannot be negative.

On the test set, Random Forest achieved a log-scale R^2 score of 0.9392. This means the model explained about 94% of the variation in dengue cases in log-transformed space. In real case-count scale, the model achieved an MAE of about 1,171 cases and an RMSE of about 6,996 cases.

The model also performed well for outbreak detection. Using an outbreak threshold of 1,106 cases, Random Forest achieved an MCC score of 0.8812, precision of 0.9253, recall of 0.9121, and F1-score of 0.9186. The high precision shows that when the model predicted an outbreak-level month, it was usually correct.

Although Random Forest achieved a slightly higher log-scale R^2 than XGBoost, its real-scale MAE and RMSE were higher. This means Random Forest captured the general log-scale pattern well, but XGBoost produced slightly better real case-count predictions.

7.4.5 Performance on Testing Data

Table 7.4: *Random Forest Performance on Testing Data*

Metric	Random Forest Result
R^2 Score in Log Scale	0.9392
Mean Absolute Error	1,171 cases
Root Mean Squared Error	6,996 cases
MAPE	41.83%
WAPE	39.13%
Accuracy-like Percentage	60.87%
MCC for Outbreak Detection	0.8812
Precision	0.9253
Recall	0.9121
F1-score	0.9186

The log-scale R^2 value shows that Random Forest learned the overall dengue trend strongly. However, the real-scale RMSE was relatively high because large dengue outbreak months create large absolute errors after inverse transformation.

7.4.6 Train-Test and Cross-Validation Performance Comparison

The Random Forest model showed stable performance in both the final train-test split and time-series cross-validation. In the final test set, the model achieved a log-scale R^2 of 0.9392, MAE of 1,171 cases, RMSE of 6,996 cases, and WAPE of 39.13%.

In 4-fold time-series cross-validation, the average Random Forest performance was also strong. It achieved a mean log-scale R^2 of 0.9309, mean MAE of 961 cases, mean RMSE of 5,358 cases, and mean WAPE of 40.60%.

The closeness between the cross-validation result and the final test result indicates that the Random Forest model generalized reasonably well. It did not depend only on a single train-test split.

7.4.7 Final Model Performance Summary

The Random Forest Regressor achieved strong performance for monthly dengue case forecasting in Brazilian states. It obtained a test-set log-scale R^2 of 0.9392, MAE of 1,171 cases, RMSE of 6,996 cases, WAPE of 39.13%, and accuracy-like percentage of

60.87%.

For outbreak detection, the model achieved an MCC of 0.8812 and an F1-score of 0.9186. These values show that Random Forest was effective at separating outbreak-level months from normal months.

Overall, Random Forest performed well because it used previous dengue trends, rolling statistics, seasonal features, climate variables, and state-level differences effectively. It worked as a strong machine learning baseline for dengue forecasting. However, compared with XGBoost, Random Forest produced slightly higher real-scale error, so XGBoost was the better model for final case-count prediction in this experiment.

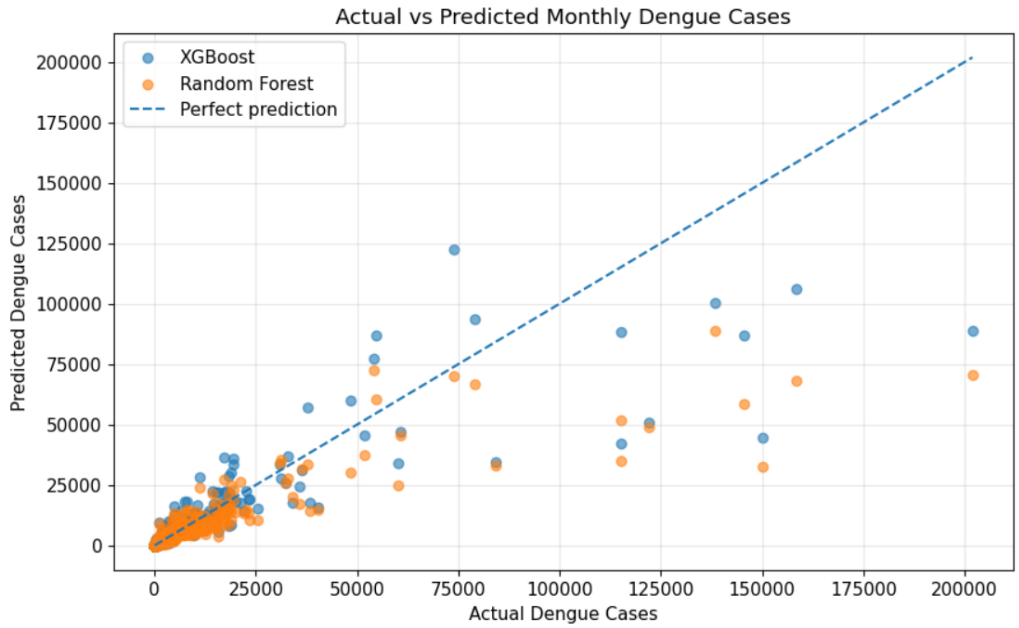


Figure 7.3

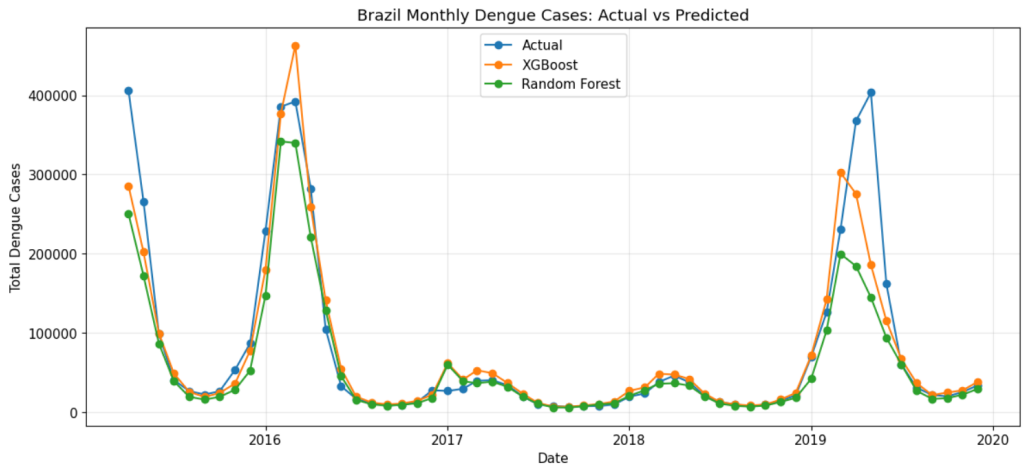


Figure 7.4

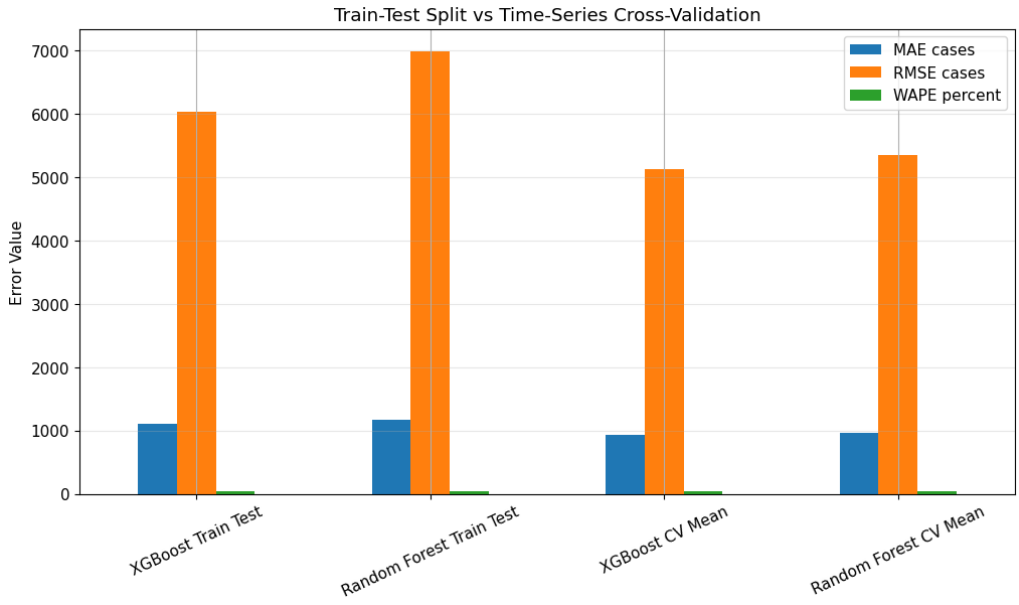


Figure 7.5

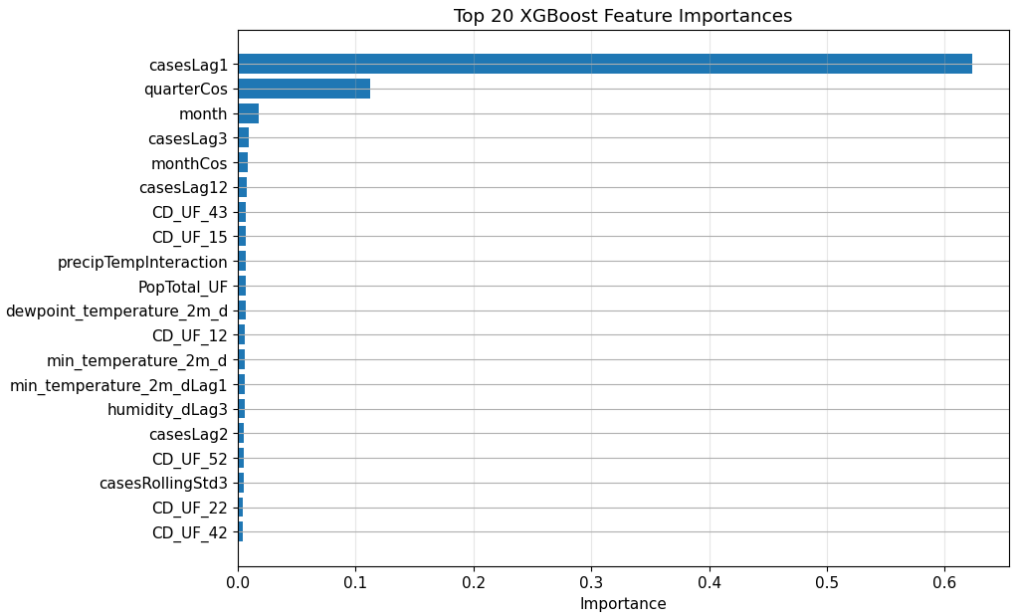


Figure 7.6

‘

7.5 Bi-Directional LSTM

A Bidirectional LSTM with Multi-Head Self-Attention is used for dengue forecasting because dengue cases follow strong time-dependent patterns. Unlike normal models, this model can learn from previous monthly records and capture how dengue cases rise and fall over time.

- LSTM helps the model remember important past information and reduce the vanishing gradient problem.
- Bidirectional LSTM processes the sequence in both forward and backward directions, giving richer temporal understanding.
- Six months of previous data are used to predict the dengue cases of the next month.
- Attention mechanism helps the model focus on the most important months in the sequence.
- Multi-head attention allows the model to learn different temporal patterns at the same time.
- The model uses dengue history, climate, and other selected features to predict log-transformed monthly dengue cases.
- The final output is a single continuous value representing the predicted dengue case count.

Overall, the Bi-LSTM with attention is suitable for dengue forecasting because it can capture seasonality, previous outbreak trends, and important temporal signals more effectively than simple tabular models.

7.5.1 Bidirectional LSTM Hyperparameters

- The model used two stacked LSTM layers. This gave enough depth to learn temporal dengue patterns, while a third layer increased training time without improving validation performance.
- The hidden dimension was set to 128. Since the LSTM is bidirectional, it produces a 256-dimensional output, which is projected back to 128 before the attention layer.
- The model was trained using the AdamW optimizer with an initial learning rate of 0.001.
- A ReduceLROnPlateau scheduler was used to reduce the learning rate when validation loss stopped improving. The minimum learning rate was set to 1e-6.

- Dropout of 0.2 was applied between LSTM layers, inside the attention module, and in the regression head to reduce overfitting.
- The attention module used 4 attention heads. Each head learned a different temporal pattern from the 128-dimensional representation.
- The model used Huber loss with delta = 1.0 as the objective function. This helped reduce the effect of very large dengue outbreak values.
- The random seed was fixed at 42 for Python, NumPy, and PyTorch to make the results reproducible.

.

7.5.2 Machine Learning Pipeline

The pipeline proceeds through seven stages. Raw monthly dengue data for 27 Brazilian states (2002 to 2019) is loaded and column-validated. Feature engineering then produces all lagged variables, rolling statistics, cyclical encodings, and demographic features. The data is split chronologically into training (years prior to 2016), validation (2016 to 2017), and test (2018 to 2019) partitions. Separate StandardScalers for features and target are fitted on training data only and applied across all three sets. Overlapping six-month sequences are built within each state's continuous time series, with partition assignment determined by the year of the target month. These sequences are loaded via PyTorch DataLoaders with a batch size of 128 and shuffle enabled for training. Finally, the trained model generates predictions on the test set, which are inverse-transformed and evaluated against actual case counts.

7.5.3 Model Training Procedure

Training began with PyTorch default weight initialisation and AdamW optimiser settings as described above. Gradient norms were clipped at 1.0 per update to prevent the exploding gradient instability common in recurrent networks. At epoch 1, training Huber loss stood at 0.1781 and validation loss at 0.1660. Early stopping with a patience of 30 epochs triggered at epoch 34, well short of the 300-epoch maximum, indicating rapid convergence. The best recorded validation loss was 0.0607. The weights from that epoch were restored for all subsequent evaluation. The fast convergence reflects the strength of the autoregressive lag features as near-term predictive signals that the LSTM can exploit from very early in training.

7.5.4 Model Testing and Results

- After training, the best Bi-LSTM with Multi-Head Self-Attention model was selected using the lowest validation Huber loss.
- The best model was found at epoch 34, with a validation Huber loss of 0.0607.
- The trained model was tested on 648 unseen test sequences from January 2018 to December 2019 across all 27 Brazilian states.
- Each test sample used six months of previous feature history to predict the dengue case count for the next month.
- The test data was not used during training or validation, so it provides a fair evaluation of the model.
- In the log-transformed scale, the model achieved:
  - R^2 score: 0.8241
  - MAE: 0.6348
  - RMSE: 0.8272
- The R^2 value of 0.8241 shows that the model explained more than 82% of the variation in dengue cases in log scale.
- The predicted values were converted back to actual dengue case counts using inverse scaling and exponential transformation.
- After inverse transformation, negative predictions were clipped to zero because dengue case counts cannot be negative.
- The Pearson correlation between actual and predicted case counts was 0.8226, showing that the model followed the general rise and fall of dengue cases well.
- The model could predict low values for off-season months and higher values for outbreak-prone months, although exact outbreak magnitudes may still have some error.
- The test period included both a quieter dengue year in 2018 and a stronger resurgence in 2019, making it a useful period for testing forecasting ability.
- During training, the loss decreased from 0.1781 at epoch 1 to about 0.0607, showing that the model learned effectively.
- Since training and validation losses remained close, the model did not show severe overfitting.
- Overall, the Bi-LSTM with attention performed well for monthly dengue forecasting by learning temporal trends, seasonal patterns, and previous outbreak behavior.

7.5.5 Performance on Testing Data

Table 7.5: *Bi-LSTM with Attention Performance on Testing Data*

Metric	Log Scale	Real Scale
R^2 Score	0.8241	0.5190
Mean Absolute Error	0.6348	1,760 cases
Root Mean Squared Error	0.8272	8,220 cases
MAPE	N/A	107.84%
Pearson Correlation	N/A	0.8226

7.6 Train-Test Performance Comparison

The closeness of training and validation Huber losses at convergence, combined with an out-of-sample log-scale R^2 of 0.8241 on the 2018 to 2019 test set, indicates acceptable generalisation from the 2001 to 2015 training period to the unseen test years. The reduction in R^2 from log space (0.8241) to real space (0.5190) is not a sign of overfitting but rather reflects the variance amplification inherent in the expm1 back-transformation, which magnifies log-scale errors in proportion to the true case count at each data point.

7.7 Final Model Performance Summary

The Bidirectional LSTM with Multi-Head Self-Attention achieves a test-set R^2 of 0.8241 in log space and 0.5190 in real case-count space, with an MAE of 1,760 cases, RMSE of 8,220 cases, and Pearson correlation of 0.8226 across 648 state-month test observations spanning 2018 to 2019. The model converged at epoch 34 with best validation Huber loss of 0.0607 and shows no evidence of severe overfitting. These results establish the Bi-LSTM with attention as a competitive sequential deep learning baseline for monthly state-level dengue forecasting in Brazil, providing a meaningful reference point for comparison with the Temporal Fusion Transformer evaluated in the following chapters.

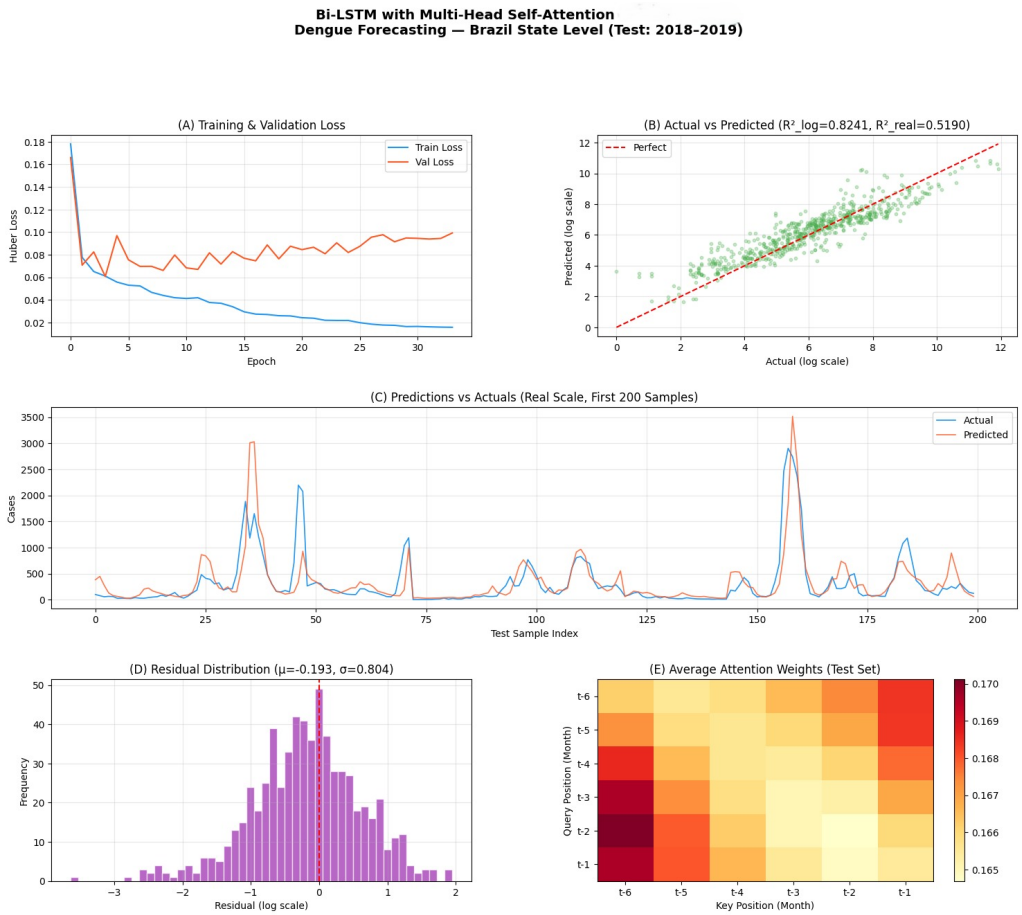


Figure 7.7

7.8 Temporal Fusion Transformer (TFT)

The Temporal Fusion Transformer (TFT) is used for dengue forecasting because it can handle different types of time-series features together. It works well when the dataset contains both static features and time-varying features.

In this project, TFT predicts log-transformed monthly dengue cases using a 12-month lookback window. This means the model uses the previous 12 months of data to predict the dengue case count for the next month.

The model uses:

- Static features: State-level information such as population, area, HDI, forest cover, and urban cover.
- Temporal features: Previous dengue cases, rainfall, temperature, humidity, NDVI, month encoding, and year information.
- Variable Selection Network: Helps the model automatically focus on the most impor-

tant features.

- LSTM encoder: Learns the time-based dengue pattern from the 12-month sequence.
- Multi-head attention: Helps the model focus on the most important months in the input window.
- Final regression layer: Produces one continuous output value, which is the predicted dengue case count.

The TFT is suitable for dengue forecasting because dengue depends on both long-term state characteristics and monthly changing conditions. The 12-month window also helps the model capture yearly seasonality more effectively than shorter sequence models.

7.8.1 Temporal Fusion Transformer Hyperparameters

The model dimension was set to 64.

The attention module used 4 attention heads.

The LSTM encoder used 1 layer.

The model was trained using the AdamW optimizer.

The learning rate was 0.0005.

Dropout was set to 0.15.

The loss function was Huber loss with delta = 1.0.

Gradient clipping was applied with a maximum norm of 1.0.

The random seed was fixed at 42.

Additional experiments were performed using multiple seeds: 42, 123, 456, 789, and 1024.

7.8.2 Model Training Procedure

Training began with PyTorch default weight initialisation and the AdamW optimiser at learning rate 5e-4 with weight decay 1e-4. Gradient norms were clipped at 1.0 per step. At epoch 1, training loss was 0.1188 and validation loss was 0.0788. The ReduceLROnPlateau scheduler halved the learning rate when validation loss stalled for 10 consecutive epochs, down to a minimum of 1e-6. Early stopping with patience of 35 epochs triggered at epoch 42, at which point the best validation Huber loss of 0.0607 was recorded. The best weights were restored before all subsequent evaluation. Notably, TFT and the Bi-LSTM converged to the same best validation loss of 0.0607 despite their architectural differences, suggesting that the Huber validation loss reaches a floor set by the irreducible

unpredictability of monthly state-level dengue incidence, and that the architectural advantages of TFT manifest primarily in better use of the available predictive signal rather than in a lower absolute loss floor.

7.8.3 Model Testing and Results

- After training, the best TFT model was restored from epoch 42, where the validation Huber loss was 0.0607.
- The model was tested on 648 unseen test sequences from January 2018 to December 2019 across all 27 Brazilian states.
- Each test sample used:
  - 12 months of temporal features
  - 6 static state-level features
- The test data was completely separate from training and validation, so the evaluation was fair.
- In log scale, the TFT achieved:
  - R^2 score: 0.8774
  - MAE: 0.5193
  - RMSE: 0.6906
- The R^2 value of 0.8774 shows that the model explained about 87.7% of the variation in log-transformed dengue cases.
- Predictions were converted back to real dengue case counts using inverse scaling and exponential transformation.
- Negative predicted values were clipped to zero because dengue cases cannot be negative.
- The real-scale Pearson correlation was 0.8740, showing that the model followed the actual rise and fall of dengue cases well.
- Important features identified by the model included cases_lag1, cases_rolling_3, and month_sin, showing the importance of recent dengue trends and seasonality.
- The test period included a moderate dengue year in 2018 and a stronger outbreak year in 2019, making it a good period for evaluating forecasting performance.
- Training loss decreased from 0.1188 at epoch 1 to about **0.060

7.8.4 Performance on Testing Data

Table 7.6: *TFT Performance on Testing Data*

Metric	Log Scale	Real Scale
R^2 Score	0.8774	0.7151
Mean Absolute Error	0.5193	1,195 cases
Root Mean Squared Error	0.6906	6,326 cases
MAPE	N/A	74.92%
Pearson Correlation	N/A	0.8740

7.8.5 Train-Test Performance Comparison

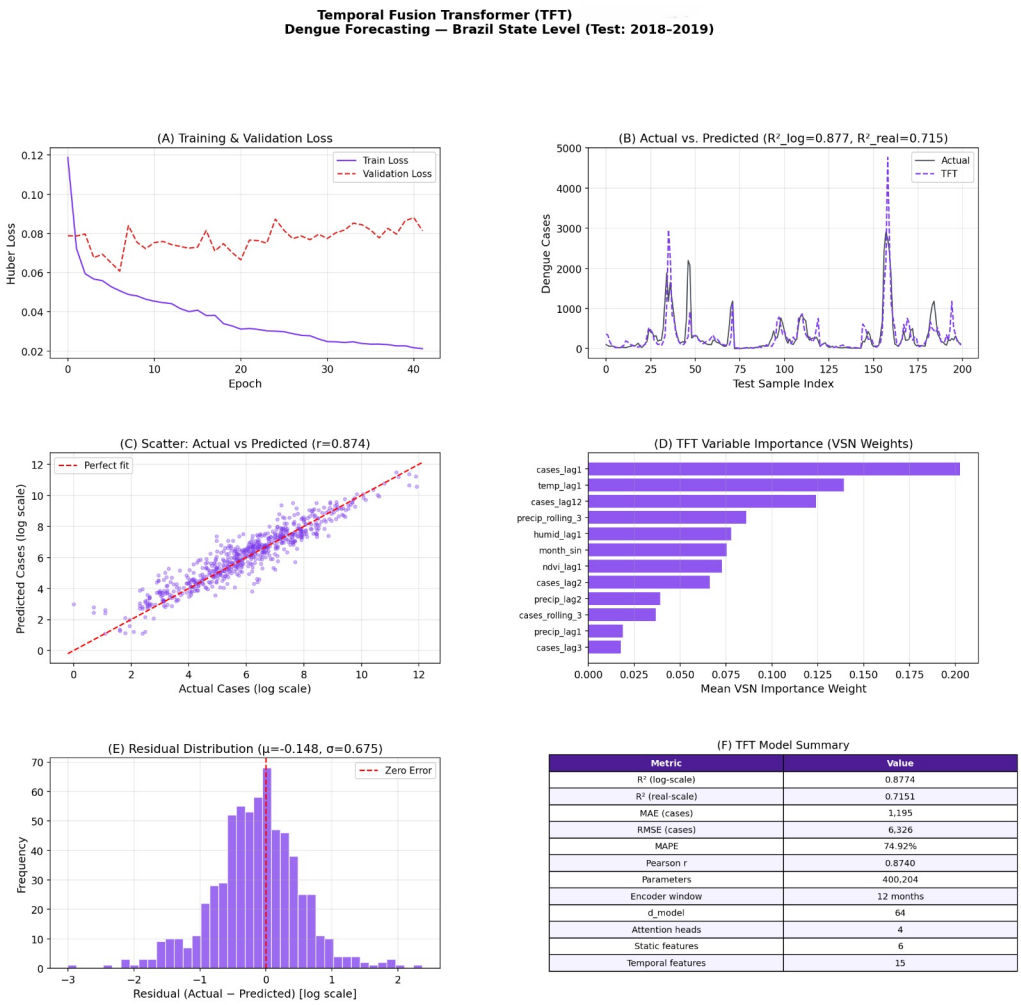
Multi-seed results across five initialisations provide the most reliable TFT performance summary. Across seeds 42, 123, 456, 789, and 1024, the model achieved a mean log-scale R^2 of 0.8645 with standard deviation 0.0144, a mean real-scale R^2 of 0.6316 with standard deviation 0.0634, a mean MAE of 1,322 plus or minus 84 cases, a mean RMSE of 7,171 plus or minus 633 cases, and a mean MAPE of 81.95% with standard deviation of 10.00%. The Pearson correlation averaged 0.8512 with standard deviation 0.0200. The relatively modest standard deviations across seeds confirm that TFT's performance advantage over the Bi-LSTM is a stable architectural property rather than a seed-dependent artefact. Comparing mean multi-seed real-scale R^2 (0.6316) to the Bi-LSTM's single-seed figure (0.5190) still shows a clear advantage for TFT, and the gap widens further when comparing the seed-42 results directly.

Table 7.7: *Bi-LSTM and TFT Performance Comparison*

Model	R^2 (log)	R^2 (real)	MAE (cases)	RMSE (cases)	MAPE (%)
Bi-LSTM + Attention	0.8241	0.5190	1,760	8,220	107.84
TFT (seed 42)	0.8774	0.7151	1,195	6,326	74.92
TFT (mean, 5 seeds)	0.8645 +/- 0.014	0.6316 +/- 0.063	1,322 +/- 84	7,171 +/- 633	81.95 +/- 10.00

7.8.6 Final Model Performance Summary

The Temporal Fusion Transformer, implemented with d_model of 64, four attention heads, a single-layer LSTM encoder, 21 total features (15 temporal plus 6 static), and a twelve-month lookback window, achieves a test-set R^2 of 0.8774 in log space and 0.7151 in real case-count space, with an MAE of 1,195 cases and RMSE of 6,326 cases across 648 state-month test observations spanning 2018 to 2019. The Pearson correlation of 0.8740 confirms strong directional accuracy. Multi-seed experiments across five initialisations yield a mean log-scale R^2 of 0.8645 plus or minus 0.014 and mean real-scale R^2 of 0.6316 plus or minus 0.063, confirming that these results are reproducible. TFT clearly outperforms the Bi-LSTM on every reported metric, demonstrating that its architectural features, including static context conditioning, the Variable Selection Network, and the longer twelve-month encoder window, translate into meaningful and reliable improvements in monthly state-level dengue forecasting accuracy for Brazil.



Metric	Value
R² (log-scale)	0.8774
R² (real-scale)	0.7151
MAE (cases)	1,195
RMSE (cases)	6,326
MAPE	74.92%
Pearson r	0.8740
Parameters	400,204
Encoder window	12 months
d_model	64
Attention heads	4
Static features	6
Temporal features	15

Figure 7.8

8 Comparison between Various Model Performance

8.1 Compare Performance of Various Models

Different models were evaluated for monthly dengue case prediction using Brazilian state-level data. The models included traditional machine learning models, deep learning models, sequence-based models, and graph-based models. Each model learned dengue patterns from previous case trends, climate variables, seasonality, population information, and state-level features.

The overall comparison shows that the Hybrid GNN + MLP achieved the highest reported (R^2) score, followed by Random Forest, XGBoost, and MLP. Among the sequential deep learning models, Temporal Fusion Transformer (TFT) performed better than Bi-LSTM with Attention, showing that TFT captured temporal and static feature relationships more effectively.

8.2 Overall Model Performance Comparison

Table 8.1: *Overall Model Performance Comparison*

Model	Model Type	Reported R^2 / Log-scale R^2	Testing Accuracy / R^2	MAE	RMSE	Main Strength
MLP	Deep Learning / Tabular	0.9194	0.9194	–	–	Learns strong local feature relationships
Hybrid GNN + MLP	Graph + Deep Learning	0.9464	0.9464	–	–	Captures both local and spatial dengue patterns
XGBoost	Machine Learning / Boosting	0.9317	0.9317	1,110 cases	6,038 cases	Best real case-count prediction among ML models
Random Forest	Machine Learning / Ensemble	0.9392	0.9392	1,171 cases	6,996 cases	Stable prediction with high log-scale R^2
Bi-LSTM + Attention	Sequential Deep Learning	0.8241	0.8241	1,760 cases	8,220 cases	Captures short-term temporal trends
TFT	Advanced Sequential Deep Learning	0.8774	0.8774	1,195 cases	6,326 cases	Best sequential model with static and temporal feature learning

8.3 Comparative Analysis

Table 8.2: *Comparative Analysis of Models*

Comparison Point	Best Performing Model	Explanation
Highest reported (R^2) score	Hybrid GNN + MLP	It combines local state-level features with spatial relationships between neighboring states.
Best traditional machine learning model	XGBoost	Although Random Forest had slightly higher log-scale (R^2), XGBoost achieved lower MAE and RMSE in real case-count scale.
Best sequential deep learning model	TFT	TFT outperformed Bi-LSTM because it used a longer 12-month window, static features, variable selection, and attention.
Best outbreak detection model	XGBoost	It achieved the highest MCC and F1-score, with very high recall for outbreak months.
Most stable baseline model	Random Forest	It produced strong performance by averaging many decision trees and reducing overfitting.
Best spatial model	Hybrid GNN + MLP	It captured dengue spread between neighboring Brazilian states, which ordinary tabular models cannot directly learn.

8.4 Final Comparison Summary

The comparison shows that each model has a different strength. The MLP performed strongly because the engineered local features were highly predictive. The Hybrid GNN + MLP gave the best overall reported (R^2), proving that spatial relationships between Brazilian states added useful information. Among machine learning models, XGBoost was the strongest final choice for real case-count prediction because it achieved lower MAE and RMSE than Random Forest. Random Forest achieved a slightly higher log-scale (R^2), but its real-scale errors were larger.

For sequential deep learning models, TFT clearly outperformed Bi-LSTM with Attention. The TFT model benefited from its 12-month lookback window, static feature processing, variable selection network, and attention mechanism. Therefore, the final comparison suggests that Hybrid GNN + MLP is the strongest spatial model, XGBoost is the strongest

machine learning model, and TFT is the strongest sequential deep learning model for monthly dengue case forecasting.

8.5 Conclusion

This project focused on predicting monthly dengue case counts for Brazilian states using machine learning and deep learning models. The dataset contained monthly dengue records from 2001 to 2019 for 27 Brazilian states, along with climate, population, environmental, geographic, and socioeconomic features. Since dengue cases are affected by previous outbreak trends, rainfall, temperature, humidity, seasonality, population size, and state-level differences, different models were applied to understand which approach performs better for dengue forecasting.

The dataset was first preprocessed by converting the date column into datetime format, sorting the records by state and time, checking missing values and duplicates, and applying proper feature engineering. Important features such as dengue lag values, rolling averages, seasonal encodings, climate lag features, population-based features, and weather interaction features were created. Data leakage was avoided by removing current-month dengue case information from the input features. A time-based train-test split was used instead of a random split so that the model evaluation could represent a realistic forecasting situation.

Six models were studied in this project: MLP, Hybrid GNN + MLP, XGBoost, Random Forest, Bi-LSTM with Attention, and Temporal Fusion Transformer. The MLP performed strongly by learning local feature relationships from tabular data. The Hybrid GNN + MLP achieved the highest reported R^2 score because it combined local state-level information with spatial relationships between neighboring Brazilian states. Among the traditional machine learning models, XGBoost and Random Forest both achieved strong performance. Random Forest showed slightly higher log-scale R^2, but XGBoost produced lower real-scale MAE and RMSE, making it more effective for real dengue case-count prediction. Among the sequential deep learning models, TFT outperformed Bi-LSTM because it used a longer lookback window, static state-level features, variable selection, and attention mechanism.

Overall, the project shows that dengue case prediction can be improved by combining previous dengue trends, seasonality, climate conditions, population features, and state-level information. The results also show that different model types have different strengths. Hybrid GNN + MLP is the strongest spatial model, XGBoost is the strongest machine learning model for real case-count prediction, and TFT is the strongest sequential deep learning model. Therefore, machine learning and deep learning approaches can be use-

ful tools for forecasting dengue cases and supporting early public health preparation in high-risk regions.